16-BIT CPU WITH SUPORT MINING

IAS-Overclockers

Ghimpău Mihai-Vladimir
Iordăchescu Vlad-Alexandru
Roșca Albert
Solomon Alexandru-Paul
Sörös Iulia
Tămaș Cosmin

Conținut

01 INTRODUCERE

02 ARHITECTURA GENERALĂ

03 DESCRIEREA MODULELOR

04 SETUL DE INSTRUCȚIUNI

05 ALU

06 MODULE MINING

07 IMPLEMENTARE ARITM. SEMNATĂ

08 STRUCTURA PROIECTULUI

09 CONCLUZII

01

INTRODUCERE

Introducere

Acest proiect implementează un procesor RISC de 16 biți cu arhitectură von Neumann, proiectat pentru aplicații embedded și operații de mining simplificat.

Introducere

Procesorul suportă:

- **Spațiul de adresare:** 1 KB RAM
- **Registre generale:** X, Y, A (acumulator)
- **Registre speciale:** PC (Program Counter), SP (Stack Pointer), AR (Address Register), DR (Data Register), IR (Instruction Register)
- **Flags:** Z (Zero), N (Negative), C (Carry), O (Overflow)
- **Caracteristici speciale:** Mining core integrat pentru operații SHA-256 simplificate

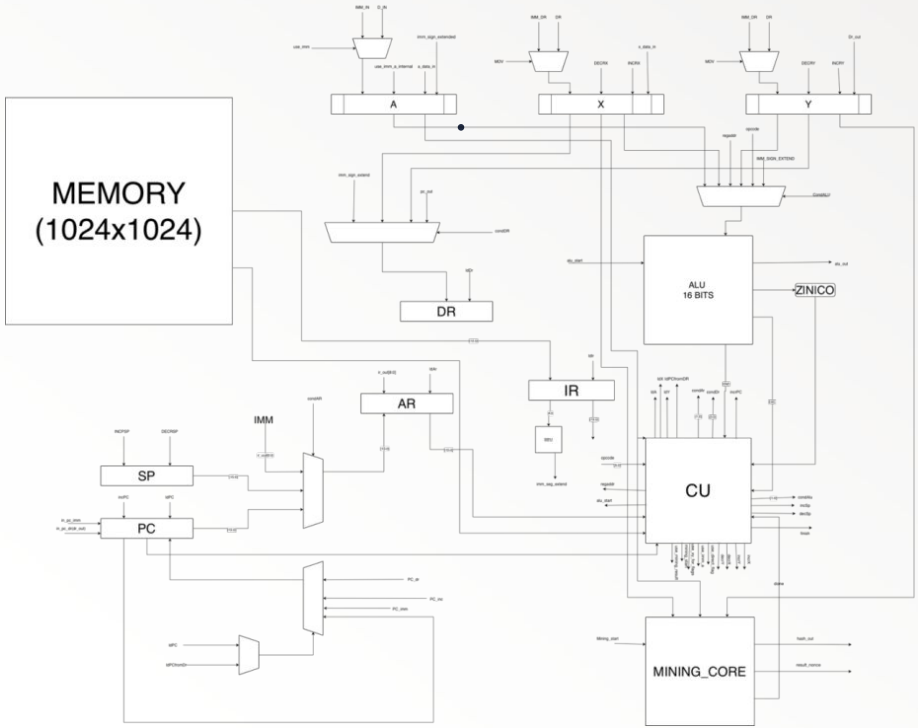
Obiectivul proiectului

Procesorul este conceput să execute operații aritmetice, logice, de control al fluxului și operații specializate de mining, oferind un echilibru între complexitate și funcționalitate pentru aplicații educaționale și demonstrative.

02

ARHITECTURA GENERALĂ

2.1. Diagrama bloc



2.2. Principii de funcționare

Ciclul Fetch-Decode-Execute:

1. **FETCH:** PC –» AR -» Memory -» DR -» IR
2. **DECODE:** CU analizează opcode-ul din IR[15:10]
3. **EXECUTE:** CU generează semnalele de control pentru execuția instrucțiunii

2.2. Principii de funcționare

Pipeline simplificat:

Procesorul implementează un pipeline de 3-5 cicluri în funcție de tipul instrucțiunii:

- Instrucțiuni simple (MOV, INC, DEC): 3-4 cicluri
- Instrucțiuni aritmetice (ADD, SUB): 4-6 cicluri
- Instrucțiuni complexe (MUL, DIV): 10-25 cicluri
- Operații de mining (MINE): 100+ cicluri

03

DESCRIEREA MODULELOR

3.1. Control Unit (CU)

Control Unit este inima procesorului, implementată ca o mașină cu stări finite (FSM) cu **47 de stări** distincte.

Stări principale

- **LOAD_ADDR**, **LOAD_INSTR**, **DECODE** - ciclul de fetch
- **HALT_STATE** - oprirea procesorului
- **LOAD_1/2/3** - încărcarea datelor din memorie
- **STORE_1/2/3** - salvarea datelor în memorie
- **BRA_1**, **BRZ_CHECK/TAKE/SKIP** - instrucțiuni de salt
- **PUSH_1/2/3**, **RET_1/2/3/4/5** - operații cu stiva
- **ALU_LOAD_OPC/OP1/OP2**, **ALU_WAIT**, **ALU_GET_RESULT** - operații ALU
- **MINE_START**, **MINE_WAIT**, **MINE_GET_RESULT** - operații de mining

Semnale de control generate

- **ldAR**, **condAR[1:0]** - control pentru Address Register
- **ldDR**, **condDR[2:0]** - control pentru Data Register
- **ldIR**, **incPC**, **ldPC**, **ldPCfromDR** - control pentru Program Counter
- **ldX**, **ldY**, **ldA** - control pentru registrele generale
- **memWR** - scriere în memorie
- **ldFLAG** - actualizare flags
- **incSP**, **decSP** - control pentru Stack Pointer
- **alu_start**, **condALU[1:0]** - control pentru ALU
- **mining_start**, **use_mining_result** - control pentru mining core

3.2. Program Counter (PC)

Registru de 16 biți care ține evidența adresei instrucțiunii curente.

Operații:

- Incrementare automată după fetch: PC = PC + 1

- Încărcare directă pentru salturi: PC = imm[9:0] (extins la 16 biți)

- Încărcare din DR pentru RET: PC = DR

- Menținere valoare pentru alte operații

Multiplexare

verilog

condPC = 00: PC_hold (menține valoarea)

condPC = 01: PC_inc (incrementare)

condPC = 10: PC_imm (salt cu valoare imediată)

condPC = 11: PC_dr (salt cu valoare din DR)

3.3. Address Register (AR)

Registru de 16 biți care ține adresa curentă pentru accesul în memorie. Doar cei 10 biți inferiori **(AR[9:0])** sunt **folosiți efectiv pentru adresare.**

Surse de adrese:

- PC - pentru fetch instrucțiuni
- SP - pentru operații cu stiva
- Valoare imediată - pentru LD/ST

3.4. Data Register (DR)

Buffer de 16 biți între memorie și procesor.

Funcții:

- Primește date din memorie (read)
- Furnizează date către memorie (write)
- Transferă date între registre (X, Y, PC)
- Buffer pentru valori imediate

3.5. Instruction Register (IR)

Registru de 16 biți care păstrează instrucțiunea curentă în execuție.

Format instrucțiune:

|OpCode|RegAddr|Data|

Opcode [15:10]: 6 biți pentru cod operație (64 instrucțiuni posibile)

RegAddr [9]: Selector registru (0 = X, 1 = Y)

Data [8:0]: Valoare imediată sau adresă

3.6. Registre generale (X, Y, A)

Registrul X și Y:

- Registre general-purpose de 16 biți
- Suportă operații de incrementare/decrementare directă
- Folosite ca operanzi pentru ALU
- X și Y pot fi folosiți pentru operații de mining (data și nonce)

3.6. Registre generale (X, Y, A)

Registrul A (Acumulator):

- Primește rezultatele operațiilor ALU
- Poate fi încărcat cu valori imediate
- Folosit ca target pentru operațiile de mining

3.7. Stack Pointer (SP)

Registru de 16 biți care indică vârful stivei, inițializat la 0x3FF (sfârșitul memoriei).

Operații:

- incSP : SP = SP + 1 (la POP/RET)
- decSP : SP = SP - 1 (la PUSH)

Convenție: Stiva crește în jos (de la adrese mari către adrese mici).

3.8. Flags Register

4 flag-uri de stare (1 bit fiecare):

- Z (Zero): 1 dacă rezultatul este 0

- N (Negative): 1 dacă bit-ul de semn (MSB) este 1

- C (Carry): 1 dacă există transport la adunare/împrumut la scădere

- O (Overflow): 1 dacă există overflow aritmetic în complement 2

3.9. Memoria RAM

Specificații:

- Capacitate: 1 KB
- Adresare: 10 biți (addr[9:0])
- Acces: Sincron cu clock
- Inițializare: din fișier data_bin.txt

Organizare:

0x000-0x03F: Zona de program (64 cuvinte)

0x040-0x3EF: Zona de date (944 cuvinte)

0x3F0-0x3FF: Zona de stack (16 cuvinte)

3.10. ALU (Unitatea Aritmetico-Logică)

ALU complexă cu suport pentru operații pe 16 biți.

Componente interne:

- **Registre:** A (17 biți cu carry), Q (16 biți), M (16 biți), Q0 (1 bit)

- **Counter-e:** CNTM (3 biți), CNTD (4 biți)

- **Unități specializate:** PBO (Parallel Bit Operations), Shifter Unit, SRT4_PLA (division)

- **Control Unit intern:** mașină de stări pentru operații complexe

3.11. Mining Core

Modul specializat pentru operații de mining bazate pe SHA-256 simplificat.

Arhitectură:

- 8 registre de stare: a, b, c, d, e, f, g, h (16 biți fiecare)
- 16 constante K[0:15] (inspirate din SHA-256)
- 16 cuvinte de mesaj W[0:15]

Funcționare

1. **INIT:** Inițializează registrele cu constante H0-H7
2. **COMPUTE:** Execută 16 runde de compresie SHA-256
3. **CHECK:** Verifică dacă hash < target
4. **DONE:** Returnează hash și nonce găsit

Formula hash:

verilog

```
hash_out = (H0 + a) ^ (H4 + e)
```

Intrări/ Ieșiri

- **data_in** : Date de hash-uit (din X)
- **nonce_in** : Nonce inițial (din Y)
- **target** : Valoarea țintă (din A)
- **hash_out** : Hash rezultat (către A)
- **result_nonce** : Nonce găsit (către X)

04

Setul de instrucțiuni

Instrucțiuni de Control

Mnemonic	Opcode	RegAddr	Data	Descriere
HALT	000000	X	xxxxxxxxx	Oprește procesorul
BRA addr	000011	X	addr[8:0]	Salt necondiționat
BRZ addr	000100	X	addr[8:0]	Salt daca Z = 1
BRN addr	000101	X	addr[8:0]	Salt dacă N = 1
BRC addr	000110	X	addr[8:0]	Salt dacă C = 1
BRO addr	000111	X	addr[8:0]	Salt dacă O = 1
PUSH	001000	X	Xxxxxxxxx	Pune PC pe stivă
RET	001001	X	xxxxxxxxx	Returnează din subrutină

Instrucțiuni de Transfer Date

Mnemonic	Opcode	RegAddr	Data	Descriere
LD r, addr	000001	R	addr[8:0]	Încarcă din memorie în r
ST r, addr	000010	R	addr[8:0]	Salvează r în memorie
MOV r, addr	011001	r	imm[8:0]	Copiază imm în r
MOVI r, addr	111001	X	imm[8:0]	Încarcă imm în A

Instrucțiuni Aritmetice (cu registru)

Mnemonic	Opcode	RegAddr	Descriere	Cicli
ADD r	001010	R	A = A + r	6
SUB r	001011	R	A = A - r	6
MUL r	001100	R	A:Q = A x r	18-24
DIV r	001101	R	Q = A / r	20-30
MOD r	001110	R	A = A % r	20-30
CMP r	010111	R	A - r	6

Instrucțiuni Aritmetice (cu valoare imediată)

Mnemonic	Opcode	Descriere	Cicli
ADDI imm	101010	A = A + imm	6
SUBI imm	101011	A = A - imm	6
MULI imm	101100	A:Q = A x imm	18-24
DIVI imm	101101	Q = A / imm	20-30
MODI imm	101110	A = A % imm	20-30
CMPI imm	110111	A - imm	6

Instrucțiuni Logice (cu registru)

Mnemonic	Opcode	RegAddr	Descriere	Cicli
AND r	010011	R	A = A & r	4
OR r	010100	R	A = A | r	4
XOR r	010101	R	A = A ^ r	4
NOT r	010110	R	A = ~r	4
TST r	011000	R	A & r	4

Instrucțiuni Logice (cu valoare imediată)

Mnemonic	Opcode	Descriere	Cicli
ANDI imm	110011	A = A & imm	4
ORI imm	110100	A = A | imm	4
XORI imm	110101	A = A ^ imm	4
NOTI imm	110110	A = ~imm	4
TSTI imm	111000	A & imm	4

Instrucțiuni de Rotate/ Shift (cu registru)

Mnemonic	Opcode	RegAddr	Descriere	Cicli
LSL r	001111	R	A « r	4+r
LSR r	010000	R	A » r	4+r
RSL r	010010	R	A rotire stanga r	4+r
RSR r	010001	R	A rotire dreapta r	4+r

Instrucțiuni de Rotate/ Shift (cu valoare imediată)

Mnemonic	Opcode	Descriere	Cicli
LSLI imm	101111	A « imm	4+imm
LSRI imm	110000	A » imm	4+imm
RSLI imm	110010	A rotire stanga imm	4+imm
RSRI imm	110001	A rotire dreapta imm	4+imm

Instrucțiuni pentru Registre

Mnemonic	Opcode	RegAddr	Descriere	Cicli
INC r	011010	R	R = R + 1	3
DEC r	011011	R	R = R - 1	3

Instrucțiuni Speciale

Mnemonic	Opcode	Descriere	Cicli
MINE	011100	Mining : găsește nonce	100-1000+

Instrucțiuni Transfer Direct între Registre

Mnemonic	Opcode	RegAddr	Operație	Utilizare Tipică
MOVR A,X	100000	0	A = X	Pregătire operand ALU
MOVR A,Y	100000	1	A = Y	Pregătire operand ALU
MOVR X,A	100001	0	X = A	Salvare rezultat ALU
MOVR A,Y	100001	1	X = Y	Transfer între registre
MOVR Y,A	100010	0	Y = A	Salvare rezultat ALU
MOVR Y,X	100010	1	Y = X	Transfer între registre

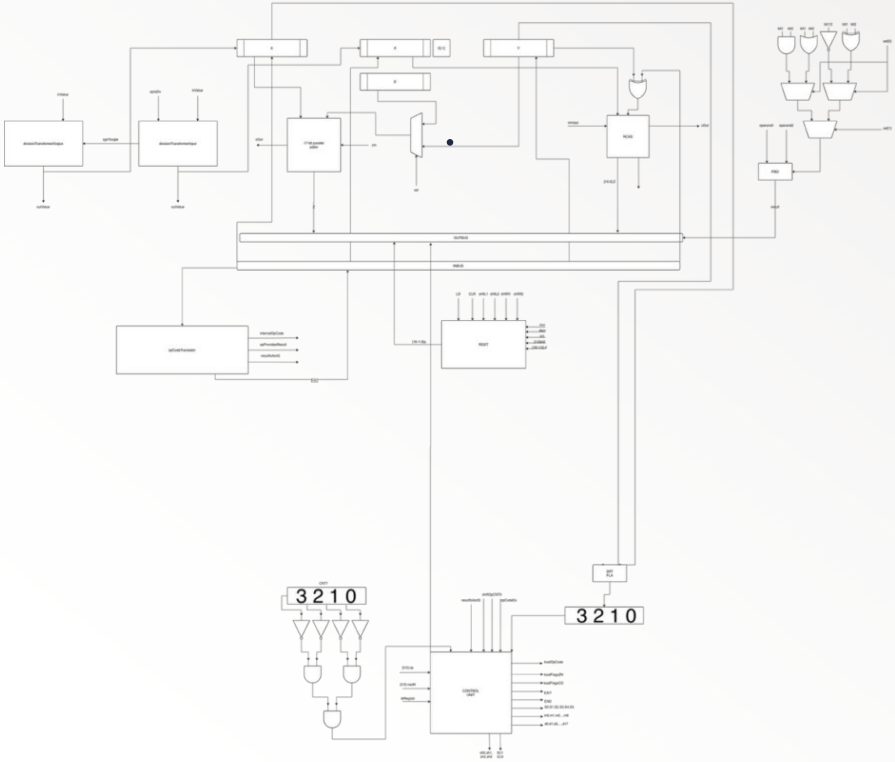
Instrucțiuni Salturi Condiționate Avansate

Mnemonic	Opcode	Condiție	Interpretare	Utilizare
BGT addr	011101	(Z=0) AND (N=O)	A > operand	Prag superior
BLT addr	011110	N ≠ O	A < operand	Prag inferior
BGE addr	011111	N = O	A ≥ operand	Minim inclusive
BLE addr	100011	(Z=1) OR (N≠O)	A ≤ operand	Maxim inclusiv

05

ALU (Arithmetic Logic Unit)

5.1. Arhitectura ALU



5.2. OpCode Translator

Convertește opcode-urile externe (de la CPU) în opcode-uri interne ale ALU: .

Operație CPU	Externel OpCode	Internal OpCode	Rezultat
ADD/ADDI	10/42	0	Reg A
SUB/SUBI	11/43	1	Reg A
MUL/MULI	12/44	2	Reg Q
DIV/DIVI	13/45	3	Reg Q
MOD/MODI	14/46	7	Reg A
LSR/LSRI	15/47	8	Reg A
LSL/LSLI	16/48	9	Reg A
RSR/RSRI	17/49	10	Reg A
RSL/RSLI	18/50	11	Reg A
AND/ANDI	19/51	12	Reg A
OR/ORI	20/52	13	Reg A
XOR/XORI	21/53	14	Reg A
NOT/NOTI	22/54	15	Reg A
CMP/CMPI	23/55	1	Doar flags
TST/TSTI	24/56	12	Doar flags

5.3. Calculul Flag-urilor

Zero Flag (Z):
verilog
Z = (A == 0) && resultIsAorQ || (Q == 0) && !resultIsAorQ

Negative Flag (N):
verilog
N = A[15] && resultIsAorQ || Q[15] && !resultIsAorQ

Carry Flag (C):
verilog
Pentru ADD/SUB
C = carryOut

Pentru MUL
C = |A[15:0] 1 dacă există overflow în registrul A

Overflow Flag (O):
verilog
Pentru ADD/SUB (complement 2)
O = cPreOut ^ cOut

Pentru MUL
O = diferit_de_extensie_semn(A, Q[15])

Pentru DIV
O = (A == 0x8000 && Q == 0xFFFF) Overflow special

5.4. Componente auxiliare

Avem 3 componente auxiliare:

- **PBO** (Parallel Bit Operations)
- **Shifter Unit**
- **RCAS** (Ripple Carry Adder/ Subtractor)

06

Modulul de mining

6.1. Descriere generală

Modulul mining_core implementează un nucleu de calcul hardware, inspirat din structura algoritmului SHA-256, dar simplificat și adaptat pentru lățimea de date de 16 biți.

Acest modul funcționează sincron, pe baza unui semnal de ceas, și este controlat printr-o mașină de stări finite (FSM), care coordonează etapele procesului de hashing: inițializare, calcul, verificare și finalizare.

6.2. Rolul modulului

Rolurile principale ale modulului sunt:

· generarea valorilor de tip *nonce*;

· calcularea unui hash pentru fiecare nonce;

· compararea hash-ului obținut cu o valoare țintă;

· semnalizarea găsirii unei soluții valide.

6.3. Interfața modulului

Semnale de intrare

· **clk** – semnal de ceas, utilizat pentru sincronizarea operațiilor;

· **reset** – semnal de reset asincron activ pe nivel logic scăzut;

· **start** – semnal de pornire a procesului de mining;

· **data_in [15:0]** – datele de intrare supuse procesului de hashing;

· **nonce_in [15:0]** – valoarea inițială a nonce-ului;

· **target [15:0]** – valoarea țintă utilizată pentru validarea hash-ului.

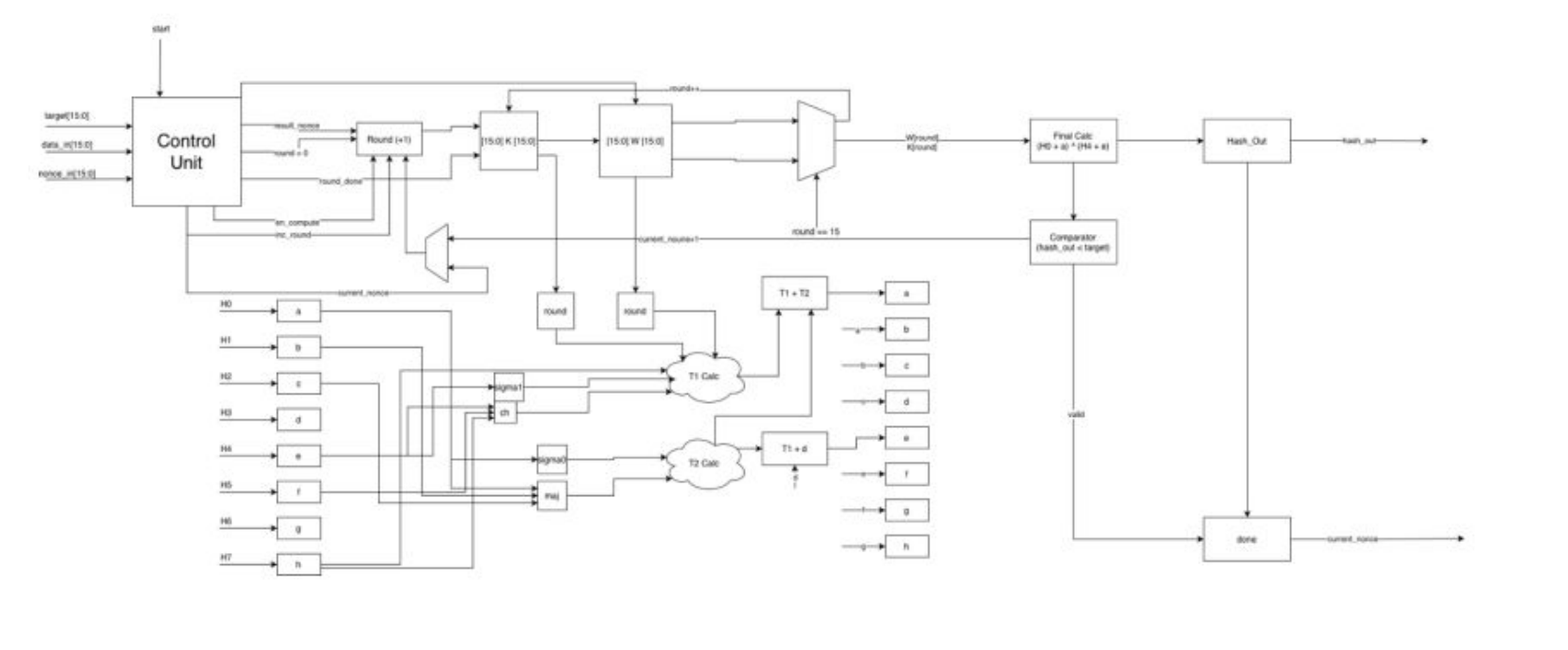
Semnale de ieșire

· **hash_out [15:0]** – rezultatul procesului de hashing;

· **result_nonce [15:0]** – nonce-ul care a generat un hash valid;

· **done** – semnal care indică finalizarea procesului de mining.

6.4. Mașina de Stări Finite (FSM)



6.5. Procesul de Hashing

Inițializarea procesului de hashing

În starea INIT, modulele interne sunt inițializate cu valori constante predefinite, similare vectorilor inițiali utilizați în algoritmul SHA-256. Registrele de lucru a–h sunt încărcate cu constantele H0–H7, iar contorul de runde este resetat.

Calculul hash-ului

În starea COMPUTE, modulul execută **16 runde de calcul**, fiecare rundă actualizând registrele interne conform următoarei logici:

· se calculează valorile intermediare T1 și T2 folosind funcții logice specifice (rotații, operații AND, XOR);

· registrele a–h sunt actualizate în mod secvențial, simulând funcționarea unei structuri de hashing de tip SHA.

La finalul ultimei runde, rezultatul hash-ului este calculat și stocat în semnalul hash_out, utilizând o combinație a valorilor intermediare obținute.

Verificarea rezultatului

În **starea CHECK**, hash-ul calculat este comparat cu valoarea țintă (target):

· dacă hash_out < target, procesul este considerat reușit, iar nonce-ul curent este salvat în result_nonce;

· dacă rezultatul nu este valid, nonce-ul este incrementat și procesul de hashing este reluat.

Această etapă reflectă principiul de bază al procesului de mining, în care **se testează succesiv diferite valori** ale nonce-ului până la găsirea unei soluții valide.

Finalizarea procesului

În starea DONE_STATE, modulul semnalizează finalizarea procesului prin activarea semnalului done. Modulul rămâne în această stare până la dezactivarea semnalului start, moment în care revine în starea IDLE, fiind pregătit pentru o nouă execuție.

Concluzie

Modulul mining_core reprezintă o implementare hardware structurată a unui algoritm de hashing de tip mining, demonstrând utilizarea registrelor interne, a funcțiilor logice complexe și a unei mașini de stări finite pentru coordonarea unui proces iterativ.

07

Implementarea Aritmeticii cu Semn

7.1. Reprezentarea Two’s Complement

Toate operațiile aritmetice folosesc **reprezentarea în complement față de 2** pentru numerele semnate:

Interval pentru 16 biți:

- Numere pozitive: 0 la 32767 (0x0000 la 0x7FFF)

- Numere negative: -32768 la -1 (0x8000 la 0xFFFF)

Conversie:

- Pozitiv → Negativ: inversare biți + 1

- Exemplu: 5 = 0x0005 → -5 = 0xFFFB

7.2. Operații de Adunare și Scădere

ADD/ADDI și SUB/SUBI:

- Adunare/scădere nativă în complement față de 2

- ALU-ul tratează toate valorile ca signed

Carry Flag (C):

- C = 1 pentru unsigned overflow (depășire la adunare)

- C = 1 pentru unsigned borrow (împrumut la scădere)

Overflow Flag (O):

- O = 1 când rezultatul depășește intervalul signed [-32768, 32767]

- Calculat: `O = carry_in_to_MSB XOR carry_out_from_MSB`

7.3. Înmulțire Semnată

Algoritm: Booth's Algorithm

- Algoritm eficient pentru înmulțire signed

- Reduce numărul de adunări necesare

Funcționare:

1. Semnul rezultatului = XOR(semn_A, semn_X/Y)

2. Înmulțire pe valori absolute

3. Aplicare semn la rezultat

Overflow Detection:

- Registrul A (17 biți cu carry) capturează overflow-ul

- C = 1 dacă există biți nenuli în A[16:15]

- O = 1 dacă rezultatul nu poate fi reprezentat pe 16 biți

7.4 Împărțire și Modulo Semnate (DIV/DIVI, MOD/MODI)

Algoritm: SRT4 (Signed Radix-4) Division

- Implementat la nivel FSM în Control Unit

- Divizor radix-4 pentru eficiență

Pipeline împărțire:

1. **D_PRE1, D_PRE2** (2 cicluri): Extragere semne, conversie la valori absolute

2. **D0-D10** (11 cicluri): Algoritm SRT4 pe valori unsigned

3. **D_POST** (1 ciclu): Aplicare semn la rezultat

7.5. Flag Updates Summary

Operații care actualizează toate flag-urile (Z, N, C, O):

- ADD, SUB, MUL, DIV, MOD (+ versiuni imediate)
- AND, OR, XOR, NOT (+ versiuni imediate)
- LSL, LSR, RSL, RSR (+ versiuni imediate)
- CMP, TST (+ versiuni imediate)

Operații care actualizează doar Z și N:

- INC, DEC
- MOV (indirect, prin valoarea mutată)

08

Structura proiectului

8.1. Organizare Directoare

```
IAS-Overclockers/
├── CPU/                    # Module Verilog principale
│   ├── cpu.v               # Top-level: integrează toate componentele
│   ├── cu.v                # Control Unit (FSM 50 stări)
│   ├── alu.v               # ALU 16-bit cu operații complexe
│   ├── mining_core.v       # Core mining SHA-256 simplificat
│   ├── memory.v            # RAM 1024x16 bit
│   ├── accumulator.v       # Registru A (acumulator)
│   ├── register_x.v        # Registru general X
│   ├── register_y.v        # Registru general Y
│   ├── program_counter.v   # PC (Program Counter)
│   ├── stack_pointer.v     # SP (Stack Pointer)
│   ├── address_register.v  # AR (Address Register)
│   ├── data_register.v     # DR (Data Register)
│   ├── instruction_register.v # IR (Instruction Register)
│   ├── seu.v               # Sign Extend Unit (9→16 bit)
│   ├── mux_ar.v            # Multiplexer pentru AR
│   ├── mux_dr.v            # Multiplexer pentru DR
│   ├── mux_alu.v           # Multiplexer pentru operanzi ALU
│   └── ...                 # Module auxiliare suplimentare
│
├── ALU/                    # ALU legacy (componente istorice)
│   └── ...                 # Fișiere de dezvoltare ALU
│
├── CPU-Assembler/          # Toolchain asamblare
│   ├── main.py             # Translator assembly → cod mașină
│   ├── opcode_table.py     # Definiții opcode-uri
│   └── ...                 # Utilități assembler
│
├── Module-Testing/         # Testbench-uri per componentă
│   ├── cpu_tb.v            # Testbench complet (58 teste)
│   ├── alu_tb.v            # Test ALU izolat
│   ├── memory_tb.v         # Test memorie
│   ├── accumulator_tb.v    # Test acumulator
│   ├── register_x_tb.v     # Test registru X
│   ├── register_y_tb.v     # Test registru Y
│   ├── data_register_tb.v  # Test DR
│   └── ...                 # Teste pentru alte componente
│
├── png_documentatei/       # Documentație și diagrame
│   ├── Documentatie_CPU.docx # Documentație completă (acest fișier)
│   ├── HW_alu_16.drawio.png  # Diagramă arhitectură ALU
│   ├── HW_cpu.drawio.png     # Diagramă arhitectură CPU
│   ├── automat_version_2.drawio.png # Diagramă FSM Control Unit
│   └── mining_core.png       # Diagramă mining core
│
├── cpu_tb.v                # Testbench simplu (register dump)
├── test_program.asm        # Program exemplu (58 instrucțiuni)
├── data_init.py            # Tool interactiv init memorie
├── data_bin.txt            # Fișier generat: cod mașină + date
├── data_bin_temp.txt       # Fișier temporar asamblare
│
├── Makefile                # Automatizare build și teste
├── run_cpu_test.sh         # Script general test cu parametri
├── run_simple_test.sh      # Script wrapper test simplu
│
├── Project_original_specs.pdf # Specificații proiect FIC
└── README.md               # Documentație utilizator (GitHub)
```

8.2. Fișiere Verilog Principale

cpu.v - Modul Top-Level
- Integrează toate componentele CPU-ului
- Conectează semnalele între module

mining_core.v - ASIP Extension
- SHA-256 simplificat (16 runde)
- Proof-of-work pentru cryptocurrency
- Interfață: data_in, nonce_in, target, hash_out

cu.v - Control Unit
- FSM cu 69 de stări
- Generează toate semnalele de control
- Coordonează ciclul fetch-decode-execute- Generează toate semnalele de control
- Coordonează ciclul fetch-decode-execute

memory.v - RAM Module
- 1024 words × 16 bits (1 KB)
- Sincron cu clock
- Inițializare din `data_bin.txt`

alu.v - Arithmetic Logic Unit
- 16 biți, operații signed/unsigned
- Componente: PBO, Shifter, RCAS, SRT4_PLA
- FSM internă pentru operații complexe (MUL, DIV)

8.3. Toolchain Files

CPU-Assembler/main.py
- Parsează fișiere `.asm`
- Convertește mnemonice în opcode-uri binare
- Generează `data_bin_temp.txt`
- Suportă labels și valori imediate

CPU-Assembler/opcode_table.py
- Definiții pentru toate cele 40+ opcode-uri
- Mapare mnemonic → cod binar 6 biți
- Referință pentru formatul instrucțiunilor

data_init.py
- Tool interactiv CLI
- Inițializează zone specifice de memorie
- Setează valori test pentru programe
- Generează/modifică `data_bin.txt`

8.4. Build și Test Files

Makefile - Automatizare Build
Targets principale:
- `make test` - Test complet (58 teste)
- `make simple` - Test simplu (dump registre)
- `make assemble` - Doar asamblare
- `make compile` - Doar compilare Verilog
- `make run` - Doar rulare simulare
- `make clean` - Curățare fișiere
- `make help` - Ajutor

run_simple_test.sh
- Wrapper pentru testare simplă
- Mai puțin verbose decât `run_cpu_test.sh`
- Util pentru iterație rapidă

run_cpu_test.sh
- Script bash general cu parametri
- Suportă program custom și testbench custom
- 5 pași: assemble → init → merge → compile → run
- Error handling și output colorat

8.5. Test Files

Module-Testing/cpu_tb.v
- Testbench complet cu 58 teste
- Verificare automată PASS/FAIL
- Acoperă toate categoriile de instrucțiuni
- Output detaliat pentru fiecare test

cpu_tb.v (root)
- Testbench simplu pentru debug
- Dump stare finală registre
- Fără verificări automate
- Rapid pentru iterații de dezvoltare

8.6. Documentation Files

README.md
- Documentație utilizator (acest conținut + mai mult)
- Format Markdown pentru GitHub
- Include exemple, comenzi, arhitectură

Documentatie_CPU.docx
- Documentație tehnică detaliată
- Diagrame arhitectură
- Specificații complete hardware
- Format academic

Project_original_specs.pdf
- Cerințe proiect FIC
- Specificații inițiale
- Criterii de evaluare

09

Concluzii

Concluzii

Proiectul demonstrează **implementarea completă și funcțională** a unui procesor educațional pe **16 biți**, cu capabilități de extindere și un accelerator hardware specializat pentru **mining criptografic**. În versiunea 2.0, procesorul implementează **39 de instrucțiuni complete**: 29 operații de bază, 9 extensii îmbunătățite (JMP, NOP, PUSH X/Y, POP X/Y, MOVR cu 6 variante, BGT, BLT, BGE, BLE) și instrucțiunea specializată **MINE** pentru **accelerare hardware**.

Procesorul îndeplinește toate obiectivele propuse:

- Arhitectură funcțională von Neumann
- Set complet de instrucțiuni RISC
- ALU cu operații aritmetice, logice și de comparație
- Algoritmi hardware avansați (Booth pentru MUL, SRT-4 pentru DIV)
- Sistem de memorie și stivă
- Unitate de control FSM robustă
- Mining core hardware cu interfață integrată
- Testare comprehensivă și validare

Concluzii Finale

Proiectul demonstrează cu succes:

1. **Fezabilitatea** unui design modular și extensibil pentru procesoare educaționale

2. **Integrarea** acceleratoarelor hardware pentru task-uri specializate

3. **Importanța** algoritmilor hardware avansați (Booth, SRT-4) pentru performanță

4. **Valoarea** testării comprehensive în dezvoltarea hardware

5. **Potențialul** pentru extinderi și optimizări viitoare

Experiența acumulată în acest proiect oferă o bază solidă pentru înțelegerea arhitecturilor moderne de procesoare și a tehnicilor de design hardware.

Anexe

Anexa A: Cod Verilog Complet

Disponibil în repository:

- `alu.v` - Modul ALU cu toate sub-modulele
- `control_unit.v` - FSM pentru control
- `opcode_translator.v` - Traducător opcode-uri
- `mining_core.v` - Accelerator mining
- `cpu_top.v` - Top-level CPU
- `memory.v` - Modul memorie
- `register_file.v` - Registre

Anexa B: Testbench-uri și Rezultate Simulări

Disponibil în repository:

- `tb_alu.v` - Teste ALU
- `tb_control_unit.v` - Teste CU
- `tb_mining_core.v` - Teste mining
- `tb_cpu_integration.v` - Teste integrare
- `waveforms/` - GTKWave waveforms

Anexa C: Script-uri de Sinteză FPGA

Disponibil în repository:

- `synth_artix7.tcl` - Script pentru Xilinx Artix-7
- `constraints.xdc` - Timing și placement constraints
- `synthesis_report.txt` - Raport resurse și timing

Anexa D: Referințe

1. **Hennessy & Patterson** - "Computer Architecture: A Quantitative Approach"

2. **Harris & Harris** - "Digital Design and Computer Architecture"

3. **Booth Algorithm** - A.D. Booth, "A Signed Binary Multiplication Technique" (1951)

4. **SRT Division** - D. E. Atkins, "Higher-Radix Division Using Estimates of the Divisor and Partial Remainders" (1968)

5. **SHA-256 Specification** - FIPS PUB 180-4